

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

Minimum cost.

$$\frac{m(m+1)}{2} + n.$$

Derivation. The cost only depends on the x being left, so take all n north steps while still at $x = 0$, where each costs 1 and they total n , then the m east steps leave from $x = 0, 1, \dots, m-1$ in turn and cost $1, 2, \dots, m$. Writing that series forwards and backwards pairs it into m columns of $m+1$, so it sums to $m(m+1)/2$.

Path. Take all n steps north from the origin first, then all m steps east.

Uniqueness. The path is unique. Any north step taken after moving east costs more than 1, since it leaves from $x > 0$, and a detour into negative x adds at least one step west and one step back east, each costing at least 1, without making any other step cheaper.

1.b

1. **False.** The state space is infinite, but UCS processes states in order of past cost and every step costs at least 1, so a state with past cost below the goal's cost is within that many steps of the origin. There are only finitely many such states, so UCS reaches the goal and terminates.
2. **False.** UCS returns the minimum-cost path, but it explores outside the rectangle: $(-1, 0)$ and $(0, -1)$ each cost 1 to reach, which is cheaper than any goal beyond the origin, so they are processed even though they lie outside $0 \leq x \leq m, 0 \leq y \leq n$.
3. **False.** UCS does return the minimum-cost path, but it can explore locations whose past cost *equals* the minimum rather than being strictly less than it. When several states tie for the lowest cost, the queue's tie-break decides the order. For the goal $(1, 0)$, which costs 1, the states $(-1, 0)$, $(0, -1)$ and $(0, 1)$ also have past cost 1 and are popped before the goal.

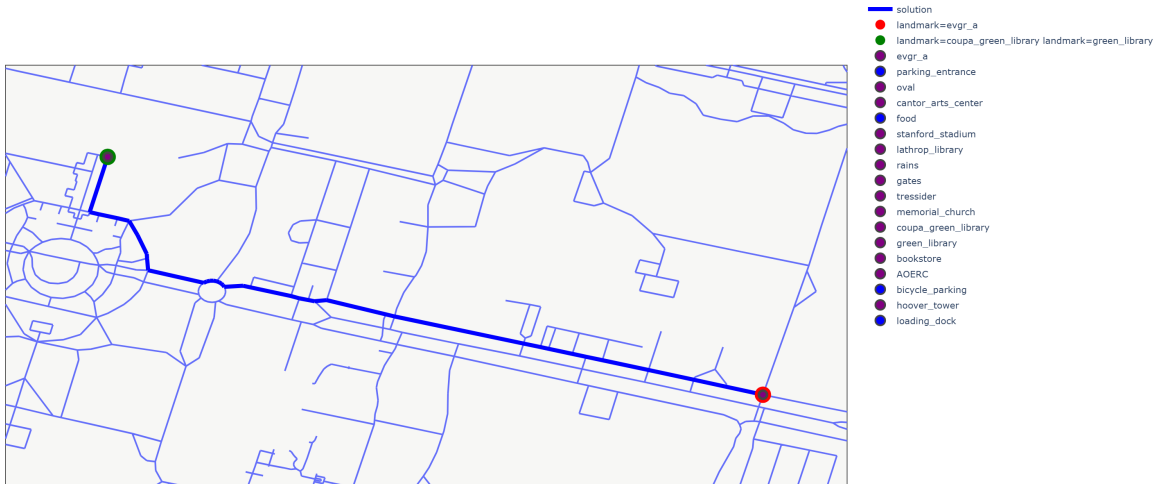
1.c

1. **True.** Adding a connection only adds paths and never removes one, so the minimum over the larger set of paths cannot exceed the minimum over the smaller set. The distance either drops, if the best path through the new connection is cheaper, or stays the same.
2. **False.** An action out of a state that the start cannot reach lies on no path from the start to the goal, however small its cost. For example, with $A \xrightarrow{5} C$ as the only route from start A to goal C and a separate component $D \xrightarrow{-1000} E$, the cheap action never appears in the minimum-cost path. The same holds for an action leading into a dead end.
3. **False.** Adding 1 to every action raises a path's cost by its number of edges, so paths with more edges are penalised more and the ranking can change. Take route 1, $A \xrightarrow{10} Z$ with one edge, and route 2, $A \xrightarrow{2} B \xrightarrow{2} C \xrightarrow{2} D \xrightarrow{2} Z$ with four. Before the change route 1 costs 10 and route 2 costs 8, so route 2 is the minimum. After adding 1 to every action, route 1 costs 11 and route 2 costs 12, so route 1 becomes the minimum-cost path.

2.b

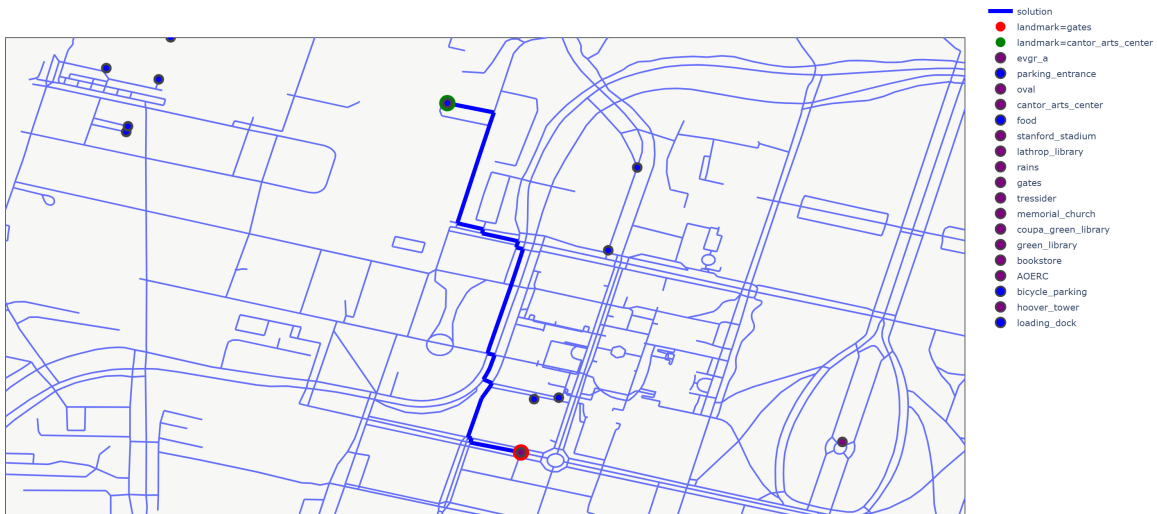
Setting 1, the useful one. Start at EVGR-A, end tag landmark=green_library. 606.0 m over 38 locations.

2b (useful): EVGR-A to Green Library



Setting 2, the one that goes wrong. Start at Gates, end tag amenity=food. 659.2 m, and it finishes at the Cantor Arts Center.

2b (undesirable): Gates to the nearest amenity=food



(i) What I learned. The cost is pure distance, so everything that actually slows a walk down is free. My EVGR-A to Green Library route is 606.0 m and passes 2 marked crossings; the Gates route is almost the same length at 659.2 m but passes 10 crossings and a bus stop. In practice the second walk takes noticeably longer, and the search has no way to tell the two apart. I also found the locations are spaced very unevenly, from 1.85 m to 98.8 m apart on the same path, so the number of steps in a path says nothing about how far it is.

(ii) What is wrong. Asking Gates for the nearest `amenity=food` returns the Cantor Arts Center at 659.2 m, while Tressider, which is the actual food court, sits at 965.9 m and never gets picked. Only four locations on the entire map carry `amenity=food`, and two of them are the art museum and the football stadium. The tag is attached to the landmark's own OSM node, so one cafe inside a large building ends up labelling the whole building as food. The modelling assumption that breaks is that a tag on a single node stands for the whole place, and that a place I can reach is a place that is open and serving.

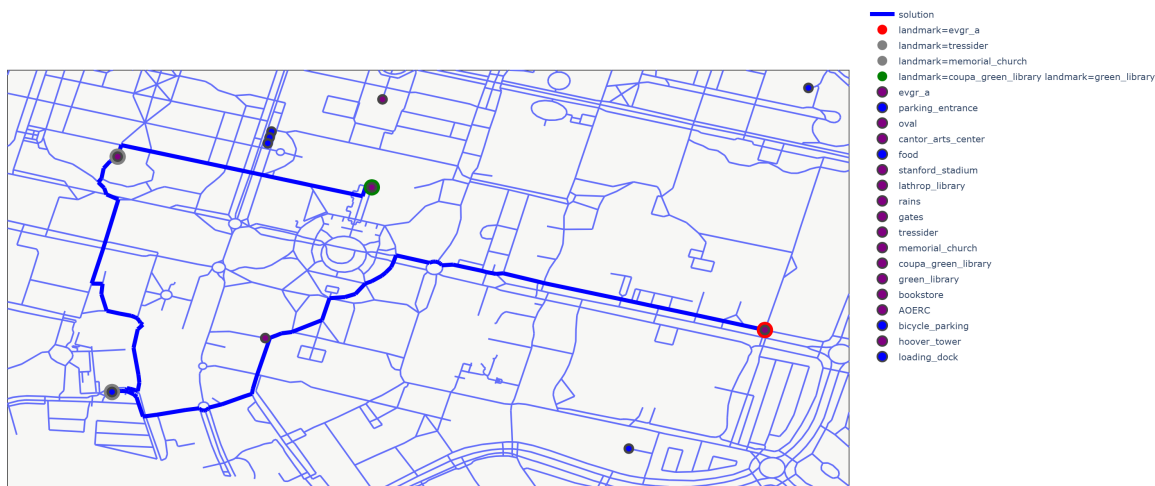
2.c

3.b

3.c

Waypoint tags. `landmark=memorial_church` and `landmark=tressider`. Start at `EVGR-A`, end tag `landmark=green_library`. Total 1815.6 m over 130 locations.

3c: EVGR-A, via Memorial Church and Tressider, to Green Library



The order surprised me. I listed the church first, but the search went to Tressider at 991.5 m, then the church at 1464.8 m, then the library at 1815.6 m. That is the point of leaving the waypoints unordered: I do not have to guess the ordering, and here my guess would have been the more expensive one. What did match my expectation is the cost. The same trip with no waypoints is 606.0 m, so adding two stops tripled it, which is what you get when the stops sit off the straight line between start and end.

What the path fails to capture is anything other than metres. It has no idea that Tressider closes in the evening, that the church is worth more time than a junction, or that I would rather eat before settling into the library than after. It is also one way, so it never accounts for getting home. A route that is optimal in distance can still be the wrong plan for the day.

3.d

5.a

5.b

6.a

6.b

6.c

6.d